Clinical Research AI Copilot

Technical architecture | Updated 4 November 2026 | Local research prototype

1. Components and data flow

```
Streamlit PDF/DOCX upload -> validation -> ordered, located document parts
  |-> original bytes -> SHA-256-addressed files + SQLite version registry
  |-> full extracted parts -> SQLite -> complete-section comparison
  `-> paragraph/token chunks -> MiniLM CPU embeddings -> persistent Chroma

Question -> predefined help OR deterministic LangGraph route + document resolution
  |-> clarification -> explicit document selection -> resume original question
  |-> exactly two versions + compare -> full-text diff -> located before/after text
  `-> lookup / summary / safety / larger compare -> scoped evidence retrieval
       -> evidence prompt -> local Ollama -> sanitize IDs -> cited-source display
```

Python 3.11 and Streamlit provide the UI. PyMuPDF handles PDFs; python-docx handles DOCX. Sentence Transformers runs all-MiniLM-L6-v2 on CPU. Ollama defaults to qwen2.5:1.5b. LangGraph exposes conditional routing; this is not an autonomous LLM agent. Packages and model weights require initial downloads, then local use needs no paid API. Configuration is centralized in `copilot/config.py`.

2. Ingestion and immutable version identity

Validate supported extension, signature, nonempty content and a 15 MB limit. PDF extraction reconstructs paragraphs, basic columns, lists and ruled tables using geometry/font cues; repeated margins are retained as typed parts but excluded from search and comparison. DOCX preserves paragraph/table order. Locations are actual PDF pages or DOCX paragraph/table references, not invented physical pages.

`copilot/versions.py` records document family, version ID, version label, unique display name, original filename, UTC upload time and SHA-256. Filename markers suggest families; explicit history selection links renamed uploads. Changed files create versions without overwriting earlier records. Identical bytes share an original, extracted parts and embeddings; a latest identical same-name upload without a label is a no-op. Fresh labels can record intentional repeats.

Default storage: Chroma vectors/text in `.chroma`; registry and full extracted parts in `.chroma/history/versions.sqlite3`; originals in `.chroma/history/sources`. `COPILOT_DB_PATH` selects another library. Back up the directory together while the app is stopped. Chroma, SQLite and file writes are not a distributed transaction.

3. Retrieval chunks and migration

`copilot/chunking.py` packs complete paragraphs within section and PDF-page boundaries. Budget: 256 tokenizer tokens including special tokens, capped by the embedding model limit; up to 32 tokens of complete trailing units overlap when space permits. Oversized paragraphs split at sentences, then words/fragments. Metadata retains section identity, paragraph ranges, token counts and PDF regions.

`copilot/index.py` lazily initializes embeddings and persists content-deduplicated chunks. Chunk-schema upgrades rebuild search chunks from retained originals while preserving comparison text. PDF extraction upgrades refresh derived parts and chunks, preserving original bytes and version records. Legacy entries without originals require re-upload for downloads/full comparison; lost overwritten content cannot be recovered.

4. Question resolution and generation

`copilot/help.py` answers standalone help/greetings locally. Other questions enter `copilot/workflow.py`. `copilot/routing.py` normalizes intent words and tolerates limited typing errors; uncertain intent can request confirmation. `copilot/resolution.py` resolves exact filenames, version markers and name tokens; ambiguous or missing explicit references request clarification before retrieval. Explicitly selected documents remain the follow-up scope.

Lookup uses cosine similarity; summary and safety add query variants and document diversity. Scoped retrieval balances evidence across selected documents and checks for missing sources. Retrieved version metadata is restored even when bytes are shared. Similarity thresholds are heuristic, not calibrated confidence. The workflow is rebuilt for each invocation; the index/model cache persists.

`copilot/llm.py` supplies only retrieved evidence to Ollama, requests `[S#]` citations, and removes unknown IDs. `copilot/citations.py` limits **Source citations** to IDs occurring in the answer, preserving IDs and showing stored filename, location, section and passage. Saved messages receive the same filtering. Similarity scores are hidden. This is citation filtering, not proof of internal model attribution, claim correctness, or exact sentence-level evidence usage.

5. Complete-version comparison

Exactly two resolved chat versions bypass retrieval and Ollama. The history view's **Compare complete versions** uses the same `copilot/comparison.py` engine. It compares full extracted sections independently of search chunk boundaries. Normalized section-body SHA-256 matches reserve unchanged content first; repeated bodies use headings, parents, neighbors and order. Remaining sections use heading and text-similarity alignment, followed by line and word diffs. Ambiguities are flagged.

The history UI shows matching method, added/removed/modified/moved content, locations, before/after text, diff, word changes and original downloads. Chat embeds source names, locations and before/after text directly. Larger chat comparisons use RAG and are not exhaustive. Images and formatting are excluded; section alignment remains heuristic. No clinical significance is inferred.

6. UI state and persistence

Messages, selected scope and pending clarification live in Streamlit session state; the library persists on disk. **New chat** clears messages, pending questions and clarification choices, and resets **Example questions** to **Choose an example...**. It preserves document selection and the library; there is no chat archive. **Deselect documents** clears scope and pending clarification while retaining messages. **Clear index and conversation** additionally deletes version history and originals.

7. Validation, limits and operating stance

Pytest covers extraction/layout, token budgets, version identity/deduplication, migration, deterministic comparison, routing, clarification, citation filtering and Streamlit chat controls. Offline fixtures use fake embeddings/tokenizers. Retrieval checks require an indexed synthetic library and matching expected filenames. Neither test success nor a citation establishes clinical correctness; manually verify answers and originals. Performance measurements are recorded separately.

No authentication, access control, study isolation, encryption-at-rest implementation or regulated audit immutability is provided. Use synthetic/authorized inputs locally. OCR, difficult PDF layouts and complex table fidelity remain gaps. Prompt warnings do not provide hardened injection protection. Priorities include hybrid search, measured retrieval quality, OCR, stronger models and reviewed security/clinical controls.

See USER_GUIDE.md for upload/version steps and TROUBLESHOOTING_NOTES.md for diagnostics.